APPFORGE / PRODUCT SPECIFICATION

Build, Export & Deploy Websites

AI Studio-ready requirements for visual editing, 3D UI, code generation, and Vercel

SOURCE REVIEWED

Pasted text.txt - 2,658 lines / 109,768 characters

STATUS NOTE

Direct user requests are separated from follow-up directions, proposals, examples, and open decisions. Referenced app ZIP contents were not verified from this text attachment.

Prepared 13 November 2026

Contents

- Visual Website Builder, 3D UI, Multi-Framework Export, and Vercel Deployment
- Instructions to the AI coding assistant
- 1. Source status and confidence labels
- 2. Product outcome
- 3. User-stated requirements
- 4. Candidate functional requirements
- 5. Technical direction proposed by the source (not binding)
- 6. Traceability and review checklist
- 7. Open questions and source conflicts
- 8. Verification expectations
- 9. Source coverage map

Instructions to the AI coding assistant

Inspect the actual project source before changing it. Preserve existing useful code and report conflicts with this brief. Use every confirmed requirement and every agreed-in-conversation direction below; do not omit, simplify, or silently reinterpret it. Keep product intent distinct from proposed implementations, illustrative content, and unresolved decisions. Do not treat sample code as production-ready without reviewing it. Do not claim a feature works unless it has been implemented and verified. Do not expose OAuth secrets or Vercel tokens in client code, browser storage, URLs, or logs. Treat generated 3D HTML/JavaScript and imported user content as untrusted: isolate previews and define a safe export/publish policy before enabling execution. If a decision is unresolved, ask the product owner or state a reversible assumption clearly. Do not invent product requirements.

1. Source status and confidence labels

The attachment is a mixed transcript, not a formal product requirements document. Use these labels when planning implementation:

- **U - User-stated:** Explicit user direction in the transcript: visual drag-and-drop website editing; users connect their own Vercel accounts; support “all code format”; implement the previously proposed core systems; offer free 3D UI assets or user-key-powered Google AI Studio 3D generation; and add features that make website creation easier. The exact boundary of “all code format” is not defined.
- **A - Accepted direction:** Features developed after the user said “yes” to assistant follow-up prompts. The user showed interest in these directions, but sometimes the question offered multiple choices and the assistant implemented multiple options. Treat the feature themes as accepted direction; treat detailed implementation choices as proposals until confirmed.
- **IP - Implementation proposal:** Suggested libraries, architecture, endpoint design, schema details, paths, file structures, storage choices, and code snippets. These are not automatically binding decisions.
- **EX - Example:** Sample presets, themes, templates, routes, values, marketing copy, and data. Illustrative only; do not ship as final content without review.
- **UQ - Unresolved:** A source ambiguity, inconsistency, security issue, or missing product decision. Do not quietly decide it.

The attachment names `app-forge-main (3).zip` and quotes an earlier assistant's claimed file structure, but also says that archive arrived as raw binary/corrupted text and that the roadmap was based on visible structure rather than a code review. This brief is based on the text attachment; it does not verify the ZIP or the current repository.

2. Product outcome

AppForge is intended to let users build and edit websites visually through drag-and-drop, use built-in or AI-generated 3D UI elements, preview the result, export a complete project in multiple code formats, and deploy it to the user's own Vercel account. The builder should reduce manual coding while still allowing advanced users to inspect generated code and inject custom CSS, JavaScript, or embed blocks.

The core product should translate a framework-neutral visual project model into buildable, framework-specific projects. Users should be able to preview, revise, save, download, and deploy their work. A user can select a target framework, connect a Vercel account, choose a new or existing Vercel project, see build progress/errors, and receive a live URL. Optional follow-up directions add multi-page sites, custom domains, global themes, templates, AI text/page generation, and user-created 3D content.

3. User-stated requirements

3.1 Visual website creation

- **U-01:** Users should build websites by drag-and-drop and edit the website to fit their requirements. No-code visual editing is the core interaction. (Source lines 66-76)
- **U-02:** Users must connect their own Vercel account to deploy. (102-105)
- **U-03:** The platform should support “all code format” / major code formats. This is explicit but unbounded; the source later enumerates static HTML/CSS/JS, React with Vite, Next.js, Vue 3, and sometimes Svelte. Treat this named list as the proposed minimum interpretation and ask what else “all” means. (102-112)
- **U-04:** The user wants the previously proposed compiler, Vercel OAuth/token, direct deploy, and cloud asset/media systems implemented in their app. (186-188, after the roadmap at 151-180)

- **U-05:** Users should have free 3D UI designs/assets available and should be able to use their own Google AI Studio API key to generate 3D UI and add it to their app. (1000-1003)
- **U-06:** Add features that make app creation easier. This is a broad direction; the concrete feature set below is from subsequent assistant proposals and short “yes” replies. (1517-1626)

3.2 Outcome expectations stated in the source

- **U-07:** Produce standard frontend projects that Vercel can recognize and build, including package metadata, build scripts, and framework entry files where applicable. (6-9)
- **U-08:** Give users a way to see and test a website before deployment. (13-15)
- **U-09:** Keep Vercel OAuth credentials and deployment calls on the backend; add authentication, rate limits, and abuse controls to reduce phishing/deployment abuse. (31-34)

4. Candidate functional requirements

4.1 Visual editor, data model, and code editing

- **CR-EDIT-01:** Provide an interactive canvas where users place, resize, and style elements without writing code. (71-76)
- **CR-EDIT-02:** Provide responsive component blocks including headers, hero sections, forms, pricing tables, and footers. Properties include colors, fonts, padding, and links. (75)
- **CR-EDIT-03:** Use a structured JSON project/tree as the editor state and source of truth for saving drafts, undo/redo, editing later, previewing, and compiling. The proposed model is framework-neutral and carries layout, styles, props, events, and state. (76, 107-112)
- **CR-EDIT-04:** Support CSS Flexbox/Grid, responsive breakpoints, global design tokens, and reusable components. (114-119)
- **CR-EDIT-05:** Allow advanced users to attach custom CSS classes, JavaScript event handlers, or raw embed blocks to visual nodes. The security boundary and exact editor UX are unresolved. (118)
- **CR-EDIT-06:** Provide a generated-code viewer with syntax highlighting and a file tree, including real-time inspection and ZIP download. Monaco Editor is a suggested implementation. (119, 155-159)
- **CR-EDIT-07:** Keep the visual canvas framework-agnostic; route one intermediate project representation to modular framework compilers. (107-112, 190-238)
- **CR-EDIT-08:** Support project/page metadata such as app name, page ID/name/slug/title, nodes, and global theme. The precise canonical schema is unresolved because several shapes appear in the source. (2052-2075)

4.2 Code generation and downloadable projects

- **CR-CODE-01:** Generate deployable, standard project files for each selected target, including valid `package.json`, build scripts, entry files, and required framework configuration/routing. Dependencies must be real and compatible. (6-9, 110-112)
- **CR-CODE-02:** Initial target formats proposed across the source: static HTML/CSS/JavaScript; React with Vite; Next.js App Router; Vue 3 with Vite. Svelte is also named as an intended target in one architecture paragraph, but no Svelte compiler or complete output set is provided. (111, 135-143, 153-159, 232-238)
- **CR-CODE-03:** Use modular compiler plugins with one common output shape: selected framework, Vercel framework preset, and a map of file paths to file contents. (190-230)
- **CR-CODE-04:** The proposed file sets are: static HTML `index.html`, `styles.css`, and `script.js`; React/Vite `package.json`, Vite config, `index.html`, `src/main.tsx`, `src/App.tsx`; Next.js `package.json`, `next.config.mjs`, `app/layout.tsx`, `app/page.tsx`; Vue 3 `package.json`, Vite config, `index.html`, `src/main.ts`, `src/App.vue`. (232-238)
- **CR-CODE-05:** Multi-page compile output should create correct route structures for static HTML, Next.js App Router, React Router/Vite, and Vue Router. Static examples use `index.html`, `about.html`, and `pricing.html`; framework examples create route/page files. (2077-2204)
- **CR-CODE-06:** Allow users to select a framework in the export/deploy UI, inspect generated files, and download a ZIP. (381-386, 846-961, 2218-2278)
- **CR-CODE-07:** Include a README in the ZIP with local run instructions and deployment guidance. (2241-2252)

- **CR-CODE-08:** Use clean and formatted code; Prettier is a proposed helper. “Error-free” and “production-ready” are goals that need measurable acceptance criteria, not claims satisfied by sample code. (8, 141)

4.3 Preview sandbox and error handling

- **CR-PREV-01:** Provide a live in-browser preview before deployment. (11-15)
- **CR-PREV-02:** Isolate preview execution so untrusted generated/user code does not run on the platform's own servers. WebContainers or Sandpack are examples, not a confirmed choice. (14)
- **CR-PREV-03:** Capture build and runtime errors in preview and provide an automated fix/retry loop before the user deploys. (15) The source does not define which errors may be auto-modified, when user approval is needed, retry limits, or rollback behavior.
- **CR-PREV-04:** Provide desktop, tablet, and mobile preview widths. Later examples name 1280 px desktop, 768 px tablet, and 375 px mobile; earlier examples do not agree on mobile width. (88-89, 1621-1625)
- **CR-PREV-05:** Render 3D widgets live in the canvas while preserving selection and drag editing. The proposal uses a `srcDoc` iframe with script-only sandbox permission and a transparent editor overlay. (1519-1597)
- **CR-PREV-06:** Avoid server-side rendering failures for browser-only 3D code: isolate browser scripts to React `useEffect`, Next.js client components, and Vue `onMounted`. (1202-1290, 1071-1073)

4.4 Vercel account connection and deployment

- **CR-DEP-01:** Register AppForge as a Vercel Integration and connect each user's Vercel account with OAuth 2.0. Request the necessary project/deployment permissions. (17-21, 121-126)
- **CR-DEP-02:** Provide a Connect Vercel control and connection status/badge. (161-167, 381-386)
- **CR-DEP-03:** Encrypt and securely store the user's access token and optional team ID in a backend token vault; never store raw tokens in plaintext or browser local storage. AES-256-GCM/PostgreSQL/Supabase is the source's proposal, not a mandated choice. (125, 161-166, 570-640)
- **CR-DEP-04:** Let the user select an existing Vercel project or create a new one; redeploy updates to the selected project. (126, 168-173)
- **CR-DEP-05:** Backend deploy flow compiles project files, hashes content using SHA-1, uploads files to Vercel, sends a deployment manifest with framework preset, and triggers a build. (24-29, 128-133)
- **CR-DEP-06:** Support Vercel framework presets proposed in the source: `nextjs`, `vite`, `vue`, or `null` for static HTML. (131, 171-172)
- **CR-DEP-07:** Show real deployment state, build errors/logs, and the final `.vercel.app` URL. Source examples poll every 3 seconds and name `QUEUED`, `BUILDING`, `READY`, `ERROR`, and `CANCELED`. The user-facing experience must handle failures and cancellation. (29, 130-133, 381-386, 808-996)
- **CR-DEP-08:** A successful deployment should expose/copy the live URL. Clipboard behavior and confirmation UI require definition. (386, 974-985)
- **CR-DEP-09:** Platform-managed hosting under one master Vercel account with wildcard subdomains is an alternative architecture in the source, but it conflicts with the user's stated preference for each user to connect their own Vercel account. Treat own-account OAuth as the intended direction; keep platform hosting out of scope unless the user changes this. (22, 94-98, 102)

4.5 Assets and media

- **CR-ASSET-01:** Store uploaded images/videos externally rather than embedding base64 or relying on local-only references, to avoid bloated/broken deploys. (82, 175-179)
- **CR-ASSET-02:** Provide an authenticated backend upload endpoint connected to a chosen cloud store: Cloudflare R2, AWS S3, or Cloudinary are options; the code example uses the S3 SDK/R2-compatible endpoint. (175-179, 343-379)
- **CR-ASSET-03:** Save returned CDN URLs into project schema through relevant editor/DataStudio controls so all generated frameworks can load assets after deployment. (178-179)
- **CR-ASSET-04:** Define upload size/type limits, file-name sanitization, ownership/access, deletion, and CDN-publication policy. The source code example omits these decisions. (UQ)

4.6 Free and AI-generated 3D UI

- **CR-3D-01:** Offer a 3D Elements category with no-cost built-in presets and the ability to add a selected preset to the canvas. “Free” must be understood as free for the user; licensing, hosting, third-party charges, and attribution still need verification. (1000-1010, 1292-1469)
- **CR-3D-02:** Include categories of 3D content proposed in the source: CSS perspective/tilt cards, glassmorphism panels, isometric buttons; external Spline scenes and GLB/GLTF models; lightweight Three.js/React Three Fiber primitives such as a torus, globe, or particle-wave background. (1005-1010)
- **CR-3D-03:** Sample built-in presets include a glassmorphism tilt card, an interactive 3D robot/model, and an isometric neon button. These names and designs are examples, not a locked catalog. (1305-1344)
- **CR-3D-04:** Let a user enter their own Google AI Studio/Gemini API key and describe a 3D UI element; generate structured output for HTML/CSS/optional JS snippets or a supported self-contained Three.js/CSS/SVG design. (1012-1049, 1371-1418)
- **CR-3D-05:** Insert generated 3D content as an editable `3d-widget` node with properties such as model URL, HTML/CSS/JS snippets, and styles. Users can resize, reposition, and edit it on canvas. (1016-1019, 1052-1069, 1401-1411)
- **CR-3D-06:** Compile supported 3D nodes for the selected export format. Load appropriate viewers for Spline and GLB/GLTF assets; keep browser-only rendering code out of server-side execution. (1052-1073, 1081-1290)
- **CR-3D-07:** Scope CSS class names for generated content (sample prefix `ai3d-`) to reduce collisions. (1385-1387)
- **CR-3D-08:** Isolate live preview in an iframe and define a robust policy for generated HTML/JS, external model URLs, scripts, and dependencies before allowing execution. An iframe sandbox proposal exists, but production CSP, sanitization, network permissions, and export behavior are unresolved. (1521-1593)
- **CR-3D-09:** AI output should be schema-validated and errors surfaced in the modal. The example schema requires `htmlSnippet` and `cssSnippet` and makes `jsSnippet` optional; validate sizes, tags, code, dependencies, and node IDs before insertion. (1021-1049, 1371-1418)
- **CR-3D-10:** User-supplied API keys are Bring Your Own Key (BYOK), so AI usage is billed/authorized through the user's provider account, not inherently free. The source suggests storing the key in localStorage or an encrypted backend; key storage policy is unresolved and the localStorage example must not automatically be treated as safe. (1014-1016, 1353-1361)

4.7 Ease-of-use features developed in the conversation

- **CR-EASE-01 - Starter templates:** One-click full-page templates for SaaS, portfolio, e-commerce storefront, and link-in-bio; also blank-canvas is mentioned earlier. (1603-1607, 1746-1916)
- **CR-EASE-02 - Section bundles:** Smart full-width sections such as Navbar + Hero + Pricing + Footer that can snap together vertically. (1606-1607)
- **CR-EASE-03 - Global theme/font controls:** Store root design tokens and apply one-click palettes across canvas and every compiler. Tokens named include background, surface, text, muted text, primary, primary text, border, radius, and font. (1609-1613, 1633-1744)
- **CR-EASE-04 - Sample theme set:** Midnight SaaS, Minimal Light, Cyberpunk Neon, and Warm Editorial are examples. (1654-1715)
- **CR-EASE-05 - AI text rewrite:** Provide an “Improve Text/Copy” action for selected headings, paragraphs, buttons, or links. Example tone choices: punchy, professional, friendly. Use the configured Gemini key; show recoverable errors. (1615-1619, 1925-2049)
- **CR-EASE-06 - Full-page AI generation:** Generate a complete responsive/themed page from one prompt, with sections such as Navbar, Hero, Features, CTA/Pricing, and Footer; optionally include one 3D hero; insert editable nodes into the canvas. (2216-2421)
- **CR-EASE-07 - Responsive layout assistance:** Automatically stack horizontal multi-column containers below 768 px unless the user overrides it; include device preview toggles. Exact responsive behavior and override UX are unresolved. (1621-1625, 1717-1737)
- **CR-EASE-08 - Multi-page support:** Add and switch between pages such as Home, About, Pricing, and Contact. Each page holds ID, name, slug, SEO title, and nodes; project holds app name, global theme, and pages. (1918-2075, 2206-2210)
- **CR-EASE-09 - Internal-link helper:** When editing a Button or Link node, provide a dropdown of existing routes so internal navigation works in all supported export formats. (2206-2210)

- **CR-EASE-10 - Local ZIP download:** Include every compiled file/folder plus a README with run/deploy steps; download a project/framework-named ZIP. (2218-2278)
- **CR-EASE-11 - Custom domains:** Let users bind a custom domain to their Vercel project, check DNS verification, display required A/CNAME records from the actual provider response, and report verified status. (2428-2655)

4.8 SEO, project storage, and platform data

- **CR-SEO-01:** Provide page title, meta description, and favicon fields; compilers inject them into the generated `<head>`. (85-89)
- **CR-DATA-01:** Persist user, Vercel connection, and project records. The suggested schema includes encrypted-token fields, team ID, target framework, project JSON, linked Vercel project name, last deployed URL, and update timestamps. Database/provider selection is unresolved. (570-609)
- **CR-DATA-02:** Define user identity/authentication and ownership boundaries for saved projects and Vercel connections. The sample schema includes user/project relationships but does not define a complete account/auth model. (580-608)
- **CR-DATA-03:** Support save/re-edit and undo/redo in the editor. The detailed persistence/history behavior is not otherwise specified in this attachment. (76)

5. Technical direction proposed by the source (not binding)

The source suggests Next.js + TypeScript + Tailwind for AppForge; Craft.js, GrapesJS, Puck, or a custom React canvas using Zustand + dnd-kit; custom AST compilers and Prettier; Node.js/Next API routes; PostgreSQL or Supabase; and Cloudflare R2, AWS S3, or Cloudinary for media. It also names WebContainers or Sandpack for isolated previews, Monaco for generated code, JSZip for downloads, Google AI Studio/Gemini with structured output for AI features, and Three.js/React Three Fiber, Spline, `<model-viewer>`, or CSS transforms for 3D.

Suggested file/folder paths include `src/utils/compilers/{types,index,helpers,htmlCompiler,reactCompiler,nextCompiler,vueCompiler,multiPageTypes,multiPageCompiler,jsx3dHelper}.ts`, `server.ts`, `server/cryptoVault.ts`, `src/components/{DeployPanel,Ai3DStudioModal,Canvas3DWidget,ThemeAndAiControls,FullPageAiGenerator,CustomDomainPanel}.tsx`, and `src/utils/ThemeEngine.ts` / `starterTemplates.ts`. These are proposed paths based on a source conversation, not verified current repository structure.

The source uses the model name `gemini-2.5-flash`, package versions, and API endpoint descriptions as examples. Verify current official docs and compatibility before implementation. The source code snippets include apparent placeholder/truncated Vercel URLs and one OAuth callback that puts an access token in a redirect query string. They are not safe or complete deployment instructions. Never copy them into production without correcting and independently verifying the endpoints, OAuth state/PKCE and callback handling, authentication/CSRF protections, server-side token retrieval, authorization, error handling, and response validation.

6. Traceability and review checklist

Status key: `[ ] User-stated` = directly expressed by user; `[ ] Accepted direction` = feature direction continued after a "yes," but details still need review; `[?] Unresolved` = must not be silently chosen; `[ ] Proposal/example` = retain as source context, not as confirmed behavior. A completion box should only be checked after implementation and verification.

ID	Requirement or decision to track	Source lines	Status
U-01	Drag-and-drop website creation and visual editing	66-76	[ ] User-stated
U-02	Users connect their own Vercel account	102-105	[ ] User-stated
U-03	"All code formats" - confirm exact supported list	102-112	[?] Scope unresolved
U-04	Add compiler, OAuth/token vault, deployment, media storage	146-188	[ ] User-stated direction
U-05	Free built-in 3D plus user-key AI 3D generation	1000-1019	[ ] User-stated
U-06	Add ease-of-use features	1517-1626	[ ] User-stated direction
EDIT-01	Canvas place, resize, style	71-76	[ ] Accepted direction
EDIT-02	Reusable visual blocks and component properties	75, 114-119	[ ] Accepted direction
EDIT-03	Framework-neutral structured JSON project model	76, 107-112	[ ] Accepted direction
EDIT-04	Flexbox/Grid, responsive breakpoints, global tokens	114-119	[ ] Accepted direction
EDIT-05	Custom CSS/JS/embed escape hatch	118	[?] Execution policy unresolved
EDIT-06	Code viewer/file tree and ZIP	119, 155-159	[ ] Accepted direction
CODE-01	HTML/CSS/JS, React/Vite, Next.js, Vue 3 outputs	111, 232-238	[ ] Accepted direction
CODE-02	Svelte output	111, 157-159	[?] Mentioned but no compiler spec
CODE-03	Framework build files and compatible packages	6-9, 110-112	[ ] Accepted direction
CODE-04	Multi-page compiler/routing for each target	82-83, 2052-2204	[ ] Accepted direction
PREV-01	In-browser isolated preview	11-15	[ ] Accepted direction
PREV-02	Capture errors and automated correction loop	15	[?] Behavior/risk unresolved
PREV-03	Desktop/tablet/mobile preview widths	88-89, 1621-1625	[?] Mobile widths conflict
PREV-04	Live 3D widget preview inside isolated iframe	1521-1597	[ ] Accepted direction
DEP-01	Vercel OAuth own-account connection	17-21, 121-126	[ ] User-stated
DEP-02	Server-side encrypted token/team storage	125, 161-166, 570-640	[ ] Accepted direction
DEP-03	Select/create Vercel project and redeploy	126, 168-173	[ ] Accepted direction
DEP-04	SHA-1 upload and deployment manifest flow	24-29, 128-133	[ ] Proposal; verify API
DEP-05	Show build state, errors/logs, final URL	29, 808-996	[ ] Accepted direction
DEP-06	Custom domains and DNS verification	2428-2655	[ ] Accepted direction
DEP-07	Platform-managed master-account hosting	22, 94-98	[?] Conflicts with own-account goal
ASSET-01	External image/video storage with CDN URLs	82, 175-179	[ ] Accepted direction
ASSET-02	Choose R2, S3, Cloudinary or other provider	82, 143, 178	[?] Provider unresolved
3D-01	Free built-in 3D library	1005-1010	[ ] User-stated direction
3D-02	CSS 3D, Spline, GLB/GLTF, Three.js primitives	1005-1010	[ ] Proposal; catalog/licensing review
3D-03	BYOK Gemini 3D generation and canvas insertion	1012-1049, 1371-1418	[ ] User-stated direction
3D-04	3d-widget schema compiled across frameworks	1052-1290	[ ] Accepted direction
3D-05	API-key storage in localStorage or backend	1014-1016, 1353-1361	[?] Security policy unresolved
3D-06	Sandbox, validate, and scope generated code	1521-1593	[?] Detailed controls unresolved

ID	Requirement or decision to track	Source lines	Status
EASE-01	Starter templates and section bundles	1603-1607, 1746-1916	[ ] Accepted direction
EASE-02	Global themes, fonts, design tokens	1609-1613, 1633-1744	[ ] Accepted direction
EASE-03	AI text rewrite, tones, selected text nodes	1615-1619, 1925-2049	[ ] Accepted direction
EASE-04	Full-page AI generation with optional 3D hero	2216-2421	[ ] Accepted direction
EASE-05	Auto-stack mobile layouts below 768 px	1621-1737	[?] Override behavior unresolved
EASE-06	Multi-page creation, page tabs, route links	2052-2210	[ ] Accepted direction
EASE-07	ZIP includes all files and README instructions	2218-2278	[ ] Accepted direction
SEO-01	Title, description, favicon injected by compiler	85-89	[ ] Accepted direction
DATA-01	Users, connections, project persistence	570-609	[?] Identity/database provider unresolved
SECURITY-01	No tokens in frontend/localStorage/query parameters	31-34, 572-640	[ ] Required security direction
SECURITY-02	Authentication, rate limits, abuse/moderation	31-34	[ ] Required security direction
DOC-01	Source code archive was not verified in this brief	146-150	[?] Review actual ZIP/repository separately

7. Open questions and source conflicts

1. **What exactly is “all code formats”?** The most detailed current list is HTML/CSS/JS, React/Vite, Next.js, and Vue 3. Svelte appears in one ambition statement, but not in the concrete `TargetFramework` type or compiler/output implementation. Confirm whether Svelte is required and whether any other targets are in scope.

2. **Which requirements ship first?** The combined scope includes core visual editing, multiple compilers, an isolated code sandbox, Vercel OAuth/deploys, asset hosting, 3D, themes, templates, AI copy/full-page generation, multi-page routing, SEO, custom domains, and token storage. The source provides no milestones or priority order.

3. **What is the canonical schema?** The source begins with a nested `ForgeNode` tree (`children`, `type`, `props`, `styles`, `text`), then describes generic JSON AST/state, then adds `3d-widget`, global themes, and a multipage `ForgePage` schema. Select one validated, versioned schema and migration plan. Keep fields consistent across all compilers and the editor.

4. **Is BYOK stored locally or server-side?** Both are suggested. Browser localStorage can expose a key to browser compromise/XSS and can persist longer than users expect. Define consent, storage, deletion, redaction, and provider billing disclosure. Do not claim user BYOK means zero cost to the user.

5. **How is user code isolated?** A sandboxed iframe is proposed for preview, while compiler examples inject AI-generated markup/scripts into final exported websites. Define permitted scripts/network access, sandbox attributes, CSP, sanitization, data boundaries, and security warnings. The preview sandbox does not by itself secure generated projects after export.

6. **How do generated project code, code edits, and visual nodes stay synchronized?** The source proposes a code viewer and direct code customization but does not explain whether code edits are one-way, parsed back into the visual schema, or allowed to detach from visual mode.

7. **What does automatic error repair change?** Set consent, scope, limits, reproducibility, diff/review, and rollback behavior before an AI loop modifies generated source automatically.

8. **Which asset service is selected?** R2, S3, and Cloudinary are alternatives. Set rules for upload ownership, media deletion, public access, file-size limits, content type validation, path safety, caching, and cost.

9. **What are the Vercel OAuth scopes and token lifecycle?** Define installation/authorization flow, CSRF/state or PKCE protections, token refresh/revocation, team/project selection, account disconnect, and deletion. The example callback places a token in a query parameter; that must not be copied.

10. **How are Vercel builds actually transferred?** Verify API endpoints, file hashing/upload protocol, framework presets, manifest schema, team query handling, logs/status API, rate limits, deployment cancellation, retries, and error mapping against

current official docs. Several sample URLs in the attachment appear truncated/placeholders.

11. **How should custom domain DNS records be shown?** Use verified Vercel API responses per domain/project; do not hardcode sample A/CNAME values. Define domain removal, duplicate domains, apex/subdomain detection, verification retries, certificate/SSL state, and errors.

12. **What counts as a free 3D asset?** Confirm licenses and attribution rules for Spline community assets, Poly Pizza, Kenney, Sketchfab, model viewers, and sample models. Free download does not guarantee rights for commercial redistribution or zero hosting/runtime costs.

13. **Which device widths are final?** The source gives 1280/768/375 and other mobile values. Confirm breakpoints and whether responsive stacking can always be overridden.

14. **Which templates/themes and product copy are approved?** Named examples (theme labels, template names, sample business text, and promotional claims) require review before shipping.

15. **What is the actual current app architecture?** The archive is named but not inspectable in this text. Inspect the real ZIP/repository before treating the cited filenames/services as existing. The earlier assistant explicitly described its prior review as structure-based rather than source-code-based.

8. Verification expectations

Before calling work complete, verify the implemented behavior against the selected scope. Evidence should cover: visual edit to schema and save/reload; each supported compiler output building with correct dependencies and routes; generated code viewer/ZIP round-trip; preview isolation and error recovery; Vercel OAuth without leaking tokens; deployment to a real connected test account with status/error reporting; upload-to-CDN references surviving compile/deploy; 3D presets and AI-generated nodes rendering safely in preview and each supported export; themes and responsive rules across targets; multi-page links, SEO metadata, and custom-domain status from real API responses. Report which environments/accounts were used and what was not verified. The source does not define release dates, performance budgets, accessibility requirements, exact account/billing plans, quotas, or support policy; do not invent them.

9. Source coverage map

- Lines 1-64: broad Vercel/code-generation/preview/security requirements and unrelated search-result snippets.
- Lines 66-145: drag-and-drop clarification, code-format scope, framework-neutral schema, editor escape hatch, Vercel OAuth, architecture/stack suggestions.
- Lines 146-188: referenced app ZIP, caveat that archive was not inspected from visible content, roadmap for compiler/OAuth/deploy/media, user asks to do all systems.
- Lines 190-386: compiler router/output files, OAuth/deploy proposal, asset-upload proposal, ExportModal and deployment status wiring.
- Lines 392-646: sample HTML/React compiler, PostgreSQL project/token tables, AES-256-GCM token vault.
- Lines 647-997: Next.js/Vue compiler proposals, status API/poller and deploy UI.
- Lines 1000-1513: user 3D request, free presets, Gemini key generator, `3d-widget` node, framework support, modal UI.
- Lines 1514-1917: live 3D iframe preview, templates/section bundles, global themes and CSS tokens, starter template samples.
- Lines 1918-2211: theme picker, AI text rewrite, multi-page schema/compiler/router/page-link controls.
- Lines 2212-2423: multi-page ZIP exporter/README and full-page AI generator with optional 3D hero.
- Lines 2424-2658: custom Vercel domain and DNS verification proposal plus sample UI/code.

Search ads/snippets near lines 39-64 are incidental content and are not product requirements. Sample model URLs, theme values, presets, names, and sample marketing copy are examples, not final content or proof of licensing.